```
################################################################################
# El Niño Potential Impact Map on Children in Southeast Asia
# Data: ERA5-Land (SPI Drought + Temp Anomaly) + WorldPop + UNICEF CCRR. Ver: Rev7.
# Author: Atiq Kainan Ahmed. Email: atiqka@yahoo.com
# ##############################################################################

# 0. SETUP ---------------------------------------------------------------

# Set this to your own project folder:
project_dir <- "/Users/atiq/Atiq_R-test/elnino26_impact"
setwd(project_dir)

# Required packages
required_packages <- c(
  "sf", "terra", "tidyterra", "ggplot2", "dplyr", "scales",
  "RColorBrewer", "rnaturalearth", "rnaturalearthdata", "readxl",
  "ecmwfr", "rnaturalearthhires"
)

missing_packages <- required_packages[
  !vapply(required_packages, requireNamespace, logical(1), quietly = TRUE)
]

if (length(missing_packages) > 0) {
  stop(
    "Please install these packages first: ",
    paste(missing_packages, collapse = ", "),
    "\nRun: install.packages(c(",
    paste(sprintf('"%s"', missing_packages), collapse = ", "),
    "))"
  )
}

library(sf)
library(terra)
library(tidyterra)
library(ggplot2)
library(dplyr)
library(scales)
library(RColorBrewer)
library(rnaturalearth)
library(rnaturalearthdata)
library(readxl)
library(ecmwfr)
library(rnaturalearthhires)


# Set up folders
dir.create("data", showWarnings = FALSE, recursive = TRUE)
dir.create("data/era5", showWarnings = FALSE, recursive = TRUE)
dir.create("outputs", showWarnings = FALSE, recursive = TRUE)

era5_dir <- file.path("data", "era5")

# CDS API key should be stored in an environment variable, not in the script.
cds_key <- Sys.getenv("MYCDSAPI")

if (!nzchar(cds_key)) {
```

```
  stop(
    "MYCDSAPI is not set. Set it in your environment, then restart R/RStudio.\n",
    "Do not paste your API key into a script you plan to share."
  )
}

# Helper: find and extract a NetCDF file from a ZIP archive safely.
unzip_if_needed <- function(name) {
  zip_path <- file.path(era5_dir, paste0(name, ".zip"))
  nc_path  <- file.path(era5_dir, paste0(name, ".nc"))

  if (file.exists(nc_path)) {
    if (file.size(nc_path) > 1024) {
      message("  Valid NetCDF already exists locally – skipping: ", basename(nc_path))
      return(invisible(nc_path))
    } else {
      message("  Existing NetCDF appears corrupted (empty/partial). Re-extracting or re-downloading...")
      file.remove(nc_path)
    }
  }

  if (!file.exists(zip_path)) {
    return(FALSE)
  }

  message("  Inspecting local ZIP archive: ", basename(zip_path))

  zip_listing <- tryCatch(unzip(zip_path, list = TRUE), error = function(e) NULL)

  if (is.null(zip_listing) || nrow(zip_listing) == 0) {
    message("  Local ZIP is corrupted or incomplete. Removing and re-downloading...")
    file.remove(zip_path)
    return(FALSE)
  }

  zip_contents <- zip_listing$Name
  nc_files <- zip_contents[grepl("\\.nc$", zip_contents, ignore.case = TRUE)]

  if (length(nc_files) == 0) {
    stop("No .nc file found inside ZIP archive: ", basename(zip_path))
  }

  selected_nc <- nc_files[1]
  unzip(zip_path, files = selected_nc, exdir = era5_dir, overwrite = TRUE)

  extracted_path <- file.path(era5_dir, selected_nc)

  if (!file.exists(extracted_path)) {
    stop("ZIP extraction did not create the expected file: ", extracted_path)
  }

  if (!identical(
    normalizePath(extracted_path, winslash = "/", mustWork = TRUE),
    normalizePath(nc_path, winslash = "/", mustWork = FALSE)
  )) {
    file.copy(extracted_path, nc_path, overwrite = TRUE)
  }
```

```
 if (!file.exists(nc_path) || file.size(nc_path) <= 1024) {
  stop("NetCDF file is invalid after extraction: ", nc_path)
 }

 message("  Successfully prepared from local cache: ", basename(nc_path))
 invisible(nc_path)
}

# Helper: download an ERA5 request with automatic retries if connection drops
download_era5 <- function(name, request, max_tries = 3) {
 nc_path  <- file.path(era5_dir, paste0(name, ".nc"))

 if (file.exists(nc_path) && file.size(nc_path) > 1024) {
  message("  NetCDF already exists locally – skipped download: ", basename(nc_path))
  return(invisible(nc_path))
 }

 if (unzip_if_needed(name)) {
  return(invisible(nc_path))
 }

 attempt <- 1
 success <- FALSE

 while (attempt <= max_tries && !success) {
  message(sprintf("  Downloading from Copernicus CDS (Attempt %d/%d): %s", attempt, max_tries, name))

  request$target <- paste0(name, ".nc")

  res <- tryCatch({
   ecmwfr::wf_request(
    request = request,
    path = era5_dir
   )
   TRUE
  }, error = function(e) {
   message("  Download attempt failed: ", conditionMessage(e))
   FALSE
  })

  if (res) {
   if (file.exists(file.path(era5_dir, paste0(name, ".zip")))) {
    unzip_if_needed(name)
   }

   if (file.exists(nc_path) && file.size(nc_path) > 1024) {
    success <- TRUE
   } else {
    message("  Downloaded file is missing or invalid. Retrying...")
   }
  }

  if (!success) {
   attempt <- attempt + 1
   if (attempt <= max_tries) {
    message("  Connection dropped or failed. Waiting 5 seconds before retrying...")
    Sys.sleep(5)
   }
```

```
   }
  }

  if (!success) {
   stop("ERA5 download failed after ", max_tries, " attempts for: ", name)
  }

  message("  Download ready and cached: ", basename(nc_path))
  invisible(nc_path)
}


# 1. STUDY AREA ----------------------------------------------------------------

asean_countries <- c(
  "Indonesia", "Philippines", "Vietnam", "Brunei", "Cambodia",
  "Laos", "Singapore", "Thailand", "Timor-Leste", "Myanmar", "Malaysia"
)

message("Loading ASEAN boundaries...")

world <- rnaturalearth::ne_countries(
  scale = "medium",
  returnclass = "sf"
) %>%
  sf::st_transform(4326)

sea_boundary <- world %>%
  dplyr::filter(
    name %in% asean_countries |
      name_long %in% asean_countries |
      admin %in% asean_countries |
      sovereignt %in% asean_countries
  ) %>%
  sf::st_make_valid()

if (nrow(sea_boundary) == 0) {
  stop("No ASEAN countries were matched in the Natural Earth boundary data.")
}

sea_vect <- terra::vect(sea_boundary)

cat(
  "Countries included:",
  paste(sea_boundary$name, collapse = ", "),
  "\n"
)


# 2. DOWNLOAD ERA5-LAND DATA (TEMPERATURE & PRECIPITATION) ---------------------

message("Checking local cache and downloading missing ERA5-Land data...")

sea_area <- c(28.5, 92, -11.5, 141)
all_months <- sprintf("%02d", 1:12)
months_2025_eval <- c("07", "08", "09", "10", "11", "12")
months_2026_eval <- c("01", "02", "03", "04", "05", "06")
```

```
# ---- A. Temperature: Evaluation Period (July 2025 – June 2026) ----

download_era5("era5_temp_2025_jul_dec", list(
  dataset_short_name = "reanalysis-era5-land-monthly-means",
  product_type       = "monthly_averaged_reanalysis",
  variable           = "2m_temperature",
  year               = "2025",
  month              = months_2025_eval,
  time               = "00:00",
  area               = sea_area,
  format             = "netcdf",
  target             = "era5_temp_2025_jul_dec.nc"
))

download_era5("era5_temp_2026_jan_jun", list(
  dataset_short_name = "reanalysis-era5-land-monthly-means",
  product_type       = "monthly_averaged_reanalysis",
  variable           = "2m_temperature",
  year               = "2026",
  month              = months_2026_eval,
  time               = "00:00",
  area               = sea_area,
  format             = "netcdf",
  target             = "era5_temp_2026_jan_jun.nc"
))

# ---- B. Temperature Climatology (2020–2025, with 2025 split) ----

for (yr in 2020:2024) {
  file_name <- paste0("era5_temp_clim_", yr)
  download_era5(file_name, list(
    dataset_short_name = "reanalysis-era5-land-monthly-means",
    product_type       = "monthly_averaged_reanalysis",
    variable           = "2m_temperature",
    year               = as.character(yr),
    month              = all_months,
    time               = "00:00",
    area               = sea_area,
    format             = "netcdf",
    target             = paste0(file_name, ".nc")
  ))
}

download_era5("era5_temp_clim_2025_h1", list(
  dataset_short_name = "reanalysis-era5-land-monthly-means",
  product_type       = "monthly_averaged_reanalysis",
  variable           = "2m_temperature",
  year               = "2025",
  month              = c("01", "02", "03", "04", "05", "06"),
  time               = "00:00",
  area               = sea_area,
  format             = "netcdf",
  target             = "era5_temp_clim_2025_h1.nc"
))

download_era5("era5_temp_clim_2025_h2", list(
  dataset_short_name = "reanalysis-era5-land-monthly-means",
  product_type       = "monthly_averaged_reanalysis",
```

```
  variable         = "2m_temperature",
  year             = "2025",
  month            = c("07", "08", "09", "10", "11", "12"),
  time             = "00:00",
  area             = sea_area,
  format           = "netcdf",
  target           = "era5_temp_clim_2025_h2.nc"
))

# ---- C. Precipitation: Evaluation Period (July 2025 – June 2026) ----

download_era5("era5_precip_2025_jul_dec", list(
  dataset_short_name = "reanalysis-era5-land-monthly-means",
  product_type       = "monthly_averaged_reanalysis",
  variable         = "total_precipitation",
  year             = "2025",
  month            = months_2025_eval,
  time             = "00:00",
  area             = sea_area,
  format           = "netcdf",
  target           = "era5_precip_2025_jul_dec.nc"
))

download_era5("era5_precip_2026_jan_jun", list(
  dataset_short_name = "reanalysis-era5-land-monthly-means",
  product_type       = "monthly_averaged_reanalysis",
  variable         = "total_precipitation",
  year             = "2026",
  month            = months_2026_eval,
  time             = "00:00",
  area             = sea_area,
  format           = "netcdf",
  target           = "era5_precip_2026_jan_jun.nc"
))

# ---- D. Precipitation Climatology year-by-year (2020–2025) ----

for (yr in 2020:2025) {
  file_name <- paste0("era5_precip_clim_", yr)
  download_era5(file_name, list(
    dataset_short_name = "reanalysis-era5-land-monthly-means",
    product_type       = "monthly_averaged_reanalysis",
    variable         = "total_precipitation",
    year             = as.character(yr),
    month            = all_months,
    time             = "00:00",
    area             = sea_area,
    format           = "netcdf",
    target           = paste0(file_name, ".nc")
  ))
}

message("ERA5 download stage completed.")


# 3. LOAD AND PROCESS CLIMATE DATA (WITH SPI PROXY) ----------------------------

message("Loading ERA5 NetCDF files and calculating SPI / Z-Score anomaly...")
```

```
temp_2025_eval_path   <- file.path(era5_dir, "era5_temp_2025_jul_dec.nc")
temp_2026_eval_path   <- file.path(era5_dir, "era5_temp_2026_jan_jun.nc")
temp_clim_files       <- c(
  file.path(era5_dir, paste0("era5_temp_clim_", 2020:2024, ".nc")),
  file.path(era5_dir, "era5_temp_clim_2025_h1.nc"),
  file.path(era5_dir, "era5_temp_clim_2025_h2.nc")
)

precip_2025_eval_path <- file.path(era5_dir, "era5_precip_2025_jul_dec.nc")
precip_2026_eval_path <- file.path(era5_dir, "era5_precip_2026_jan_jun.nc")
precip_clim_files     <- file.path(era5_dir, paste0("era5_precip_clim_", 2020:2025, ".nc"))

# Temperature Anomaly
temp_eval <- terra::rast(list(terra::rast(temp_2025_eval_path), terra::rast(temp_2026_eval_path)))
temp_clim_all <- terra::rast(lapply(temp_clim_files, terra::rast))
heat_raw <- (terra::mean(temp_eval) - 273.15) - (terra::mean(temp_clim_all) - 273.15)
names(heat_raw) <- "temp_anomaly"

# Precipitation SPI Proxy (Z-Score: [Observation - Mean] / Standard Deviation)
precip_eval <- terra::rast(list(terra::rast(precip_2025_eval_path), terra::rast(precip_2026_eval_path)))
precip_clim_all <- terra::rast(lapply(precip_clim_files, terra::rast))

precip_eval_mean <- terra::mean(precip_eval)
precip_clim_mean <- terra::mean(precip_clim_all)
precip_clim_sd   <- terra::app(precip_clim_all, fun = sd, na.rm = TRUE)

spi_raw <- (precip_eval_mean - precip_clim_mean) / (precip_clim_sd + 1e-6)
names(spi_raw) <- "spi_anomaly"


# 4. LOAD POPULATION DATA ----------------------------------------------------

message("Loading WorldPop Under 18 data...")
pop_file <- "global_t_Under_18_2026_CN_1km_R2025A_UA_v1.tif"

if (!file.exists(pop_file)) {
  stop("WorldPop raster not found: ", pop_file)
}

pop_raw <- terra::rast(pop_file)
names(pop_raw) <- "child_pop_density"


# 5. LOAD UNICEF CCRR VULNERABILITY ------------------------------------------

message("Loading UNICEF CCRR vulnerability data...")
unicef_file <- "ccrr_2026.xlsx"

if (!file.exists(unicef_file)) {
  stop("UNICEF Excel file not found: ", unicef_file)
}

vuln <- readxl::read_excel(unicef_file, sheet = "Vulnerability")

ccrr_clean <- vuln %>%
  dplyr::select(country = `Country/Territory`, vulnerability_score = `Child Vulnerability Score`) %>%
  dplyr::mutate(
```

```
   vulnerability_score = as.numeric(vulnerability_score),
   country = dplyr::recode(
    country,
    "Viet Nam" = "Vietnam",
    "Lao People's Democratic Republic" = "Laos",
    "Lao PDR" = "Laos",
    "Myanmar (Burma)" = "Myanmar"
   )
  ) %>%
  dplyr::filter(country %in% asean_countries, !is.na(vulnerability_score))

sea_boundary <- sea_boundary %>%
  dplyr::left_join(ccrr_clean, by = c("name" = "country"))


# 6. PRE-PROCESSING & HARMONIZATION ---------------------------------------------

message("Cropping and harmonising layers...")

template <- terra::crop(spi_raw, sea_vect) %>% terra::mask(sea_vect)
spi_h <- template

heat_h <- terra::crop(heat_raw, sea_vect) %>% terra::mask(sea_vect) %>% terra::resample(template, method = "bilinear")
pop_h  <- terra::crop(pop_raw,  sea_vect) %>% terra::mask(sea_vect) %>% terra::resample(template, method = "bilinear")

unicef_vect <- terra::vect(sea_boundary %>% dplyr::filter(!is.na(vulnerability_score)))
unicef_rast <- terra::rasterize(x = unicef_vect, y = template, field = "vulnerability_score", fun = "mean", background = NA)
unicef_h <- terra::resample(unicef_rast, template, method = "near")

cat("All layers successfully harmonised.\n")


# 7. MULTI-CRITERIA EVALUATION ----------------------------------------------

message("Running Multi-Criteria Evaluation...")

minmax_norm <- function(r, invert = FALSE, power = 0.4) {
  r_min <- terra::global(r, "min", na.rm = TRUE)[1, 1]
  r_max <- terra::global(r, "max", na.rm = TRUE)[1, 1]

  if (is.na(r_min) || is.na(r_max) || r_max == r_min) {
    return(r * 0)
  }

  scaled <- (r - r_min) / (r_max - r_min)

  if (invert) {
    scaled <- 1 - scaled
  }

  scaled <- scaled ^ power
  scaled * 4 + 1
}

# SPI is inverted: negative SPI (drought) = higher hazard score
spi_n    <- minmax_norm(spi_h,  invert = TRUE)
heat_n   <- minmax_norm(heat_h, invert = FALSE)
pop_n    <- minmax_norm(pop_h)
```

```
unicef_n  <- minmax_norm(unicef_h)

# Hazard Composite (50% SPI Drought Stress, 50% Temperature Heat Stress)
hazard <- (spi_n * 0.5) + (heat_n * 0.5)

# Overall Weights: Hazard (60%), Population Density (25%), Vulnerability (15%)
impact <- (hazard * 0.60) + (pop_n * 0.25) + (unicef_n * 0.15)

# Mask strictly to ASEAN boundaries (keeping internal unpopulated cells as NA)
impact <- terra::mask(impact, sea_vect)
impact <- terra::clamp(impact, lower = 1, upper = 5)

names(impact) <- "child_impact_score"


# 8. EXPORT RESULTS ------------------------------------------------------------

out_tif <- file.path("outputs", "ElNino_Child_Impact_SEA_2025_26_SPI.tif")
terra::writeRaster(impact, out_tif, overwrite = TRUE, gdal = c("COMPRESS=LZW", "TILED=YES"))
cat("GeoTIFF exported to:", out_tif, "\n")


# 9. VISUALISATION -------------------------------------------------------------

message("Creating map...")

impact_pal <- grDevices::colorRampPalette(
  c("#1a9850", "#91cf60", "#d9ef8b", "#fee08b", "#fc8d59", "#d73027", "#a50026")
)(100)

# Prepare country label coordinates safely using point on surface
sea_labels <- suppressWarnings(
  sea_boundary %>%
    dplyr::mutate(label_pt = sf::st_point_on_surface(geometry)) %>%
    sf::st_set_geometry("label_pt")
)

p <- ggplot2::ggplot() +
  tidyterra::geom_spatraster(
    data = impact
  ) +
  ggplot2::scale_fill_gradientn(
    colours = impact_pal,
    limits = c(1, 5),
    breaks = 1:5,
    labels = c(
      "1 – Lowest",
      "2 – Low",
      "3 – Moderate",
      "4 – High",
      "5 – Critical"
    ),
    name = "Potential Child\nImpact Score",
    na.value = "#EBEBEB"  # Shades unpopulated/zero-population internal cells in neutral gray
  ) +
  ggplot2::geom_sf(
    data = sea_boundary,
    fill = NA,
```

```
   colour = "grey20",
   linewidth = 0.45
  ) +
  # Add country name labels cleanly over the polygons
  ggplot2::geom_sf_text(
   data = sea_labels,
   ggplot2::aes(label = name),
   size = 3.2,
   fontface = "bold",
   colour = "grey15",
   check_overlap = TRUE
  ) +
  ggplot2::coord_sf(
   crs = 4326,
   expand = FALSE
  ) +
  ggplot2::labs(
   title = "2026 El Niño Potential Impact on Children",
   subtitle = "Southeast Asia",
   x = NULL,
   y = NULL,
   caption = paste0(
    "Hazard (SPI Drought + Temp Anomaly) |  ",
    "Child Population Density |  ",
    "UNICEF Vulnerability | ",
    "Data: Copernicus ERA5-Land, WorldPop, UNICEF CCRR  |  "
   )
  ) +
  ggplot2::theme_minimal(base_size = 12) +
  ggplot2::theme(
   plot.title = ggplot2::element_text(face = "bold", size = 15, hjust = 0.5),
   plot.subtitle = ggplot2::element_text(size = 11, hjust = 0.5, colour = "grey30"),
   plot.caption = ggplot2::element_text(size = 9.5, colour = "grey40", hjust = 0),
   legend.position = "right",
   legend.key.height = grid::unit(1.3, "cm"),
   axis.title = ggplot2::element_blank(),
   panel.grid.major = ggplot2::element_line(color = "grey75", linetype = "dashed", linewidth = 0.3),
   panel.ontop = TRUE
  )

# Generate timestamped filename
timestamp_str <- format(Sys.time(), "%Y%m%d_%H%M%S")
map_png <- file.path("outputs", paste0("ElNino_Child_Impact_Map_SEA_2025_26_", timestamp_str, ".png"))

# 1. Save high-resolution PNG to outputs folder
ggplot2::ggsave(
  filename = map_png,
  plot = p,
  width = 12,
  height = 9,
  dpi = 300,
  bg = "white"
)

# 2. Force an active graphics window to open and plot live in the RStudio panel
if (interactive()) {
  grDevices::dev.new(width = 12, height = 9)
}
```

```
print(p)

cat("\n=== Script completed successfully ===\n")
cat("Map successfully saved to:", map_png, "\n")


# End of script - Revised 7 #
```